In the rest of this section we describe the different proof automation techniques used in this project. We first discuss the different proof methods used: primarily superposition and equational reasoning; we then discuss the integration in *Lean*, and finally we report some basic empirical results from this project.

7.1. **Proof techniques.** The two main families of ATPs and tactics we used are based on superposition/saturation and equational reasoning. In this context we also include SMT solvers, which combine specific decision procedures for theories, like congruence closure for equational reasoning, with satisfiability (SAT) solving [19]. Finally, we also used `aesop` [41], which implements a version of tableau search. This was used mainly to help specific constructions in refutations, and is not specific to proving or disproving magma implications in this sense. We describe our use of `aesop` in Section 7.2 below.

7.1.1. *Saturation.* Most of the ATPs used extensively in this project rely primarily on saturation procedures in the superposition calculus. For example, this is the case for *Vampire* [38].[10] The core idea of these provers is that they take a set of assumptions and a conjecture, expressed in (say) first-order logic. The conjecture is negated and added to the set of assumptions, which are all put into a normal form. The ATP then tries to refute the negation by applying rules of an underlying calculus, until a proof of false (a contradiction) is derived. In this case, the conjecture was (classically) true, and the ATP has found a proof by contradiction, often called a "refutation" or "saturation" proof.

The underlying calculi vary from system to system, but they often have a variant of a resolution clause of the form:

$$\frac{C \vee L \quad D \vee \neg L}{C \vee D}$$

This can be read as $C \vee L$ with $D \vee \neg L$ implies $C \vee D$, where C, D, L are formulas in e.g. first-order logic. Superposition calculi have a variant of this rule that deals with equality directly, and thus are more efficient at reasoning about equality.

In this project we used *Vampire* [38], *Duper* [17] and *Prover9* and *Mace4* [43] which are all based on variants of saturation for proving.

7.1.2. *Equational Reasoning.* As already discussed in Section 6.3, equational reasoning is a type of reasoning that is based[11] on equational logic and rewriting with congruence [7]. In general, an equational reasoning procedure takes a series of equations and tries to determine whether another equation can be deduced from it. A core tool in equational reasoning are *e-graphs*, a data structure used to represent congruence classes of terms. By themselves, e-graphs provide an efficient means of implementing a decision procedure for congruence closure over ground equations (i.e., equations without variables). Extensions to this procedure, for example by quantifier instantiation via e-matching [20], also allow for a semi-decision procedure for congruence closure over non-ground equations.

[10] See also [9] for a gentler exposition.

[11] More precisely, one can formalize this reasoning using Birkhoff's five rules of inference (reflexive, symmetric, transitive, replacement, and substitution); see, e.g., [14].

SMT solvers like *Z3* [21] use equational reasoning for deciding the theory of equality with uninterpreted functions [39, 20]. On the other hand, equality saturation [69] uses e-graphs by extending congruence closure to a more controlled search, enabling optimization and conditional rewriting. One of the main advantages of equational reasoning for implications of magma laws is that we get very explicit proofs: a proof that $l \models l'$ is given by a sequence of rewrites that starts at the left-hand side of l' and arrives at the right-hand side through applications of l.

In this project we used *Z3* [21], *Prover9* and *Mace4* [43], a custom ATP *MagmaEgg* for magmas based on *egg* [69], and the *Lean* `egg` tactic [37, 60], which all work with equational logic. We have also reasoned with manual (custom written) heuristics about simple rewrites.

7.2. **Integration of automation procedures.** While ATPs are very useful for proving theorems in this project, they do not integrate with *Lean* out of the box. ATPs may produce unsound proofs, or worse, derive incorrect results. Thus, by default, theorems in *Lean* cannot be proven by deferring to the result of an ATP. Instead, the results of an ATP can be used to reconstruct a proof of the form required by *Lean*. Thus, in general, integration of ATPs requires two steps. First, there is the invocation of the ATPs by translating the problem from *Lean* into the languages and logics they use. And second, there is the reconstruction of the ATPs' results as a (persistent) *Lean* proof. These two aspects present different challenges, and require different strategies, depending mostly on the kind of proof strategy the ATP uses.

More generally, we have observed that there are multiple ways of integrating decision procedures within *Lean*, with different levels of integration.

(1) Using a *Lean* tactic, which calls a decision procedure written in *Lean* (like `aesop` or `duper`).
(2) Using a *Lean* tactic, which calls an existing (external) ATP and reconstructs a proof term from the ATP's result (like `bv_decide` or `egg`).
(3) Using an external script which calls an existing ATP and generates a source file `.lean` which captures the result explicitly.

This project primarily used the least integrated approach, Option 3, as it was the fastest to implement and imposed no additional technical requirements on other contributors. The matter of technical requirements caused problems, for example, when integrating the `egg` tactic (Option 2) as it initially expected certain software on the user's machine. Such trade-offs between Option 2 and Option 3 are, however, mutual, as the higher upfront cost of integrating a proof tactic in Option 2 makes the decision procedure easier to use than with Option 3. Additionally, Option 2 can benefit from *Lean*'s meta-programming capabilities when encoding the problem for use with an ATP, and when reconstructing a *Lean* proof from the result.

7.2.1. *Proof Reconstruction.* The relative simplicity of the objects used in this project benefits the implementations of proof reconstruction. By focusing on the given problem domain, difficult reconstruction issues, like complex dependent types, could be ignored.

```
@[equational_result] theorem Equation999_implies_Equation86
    (G : Type*) [Magma G] (h : Equation999 G) : Equation86 G := by
  by_contra nh
  simp only [not_forall] at nh
  obtain ⟨sK0, sK1, sK2, nh⟩ := nh
  have eq9 (X0 X1 X2 X3 : G) : (X1 ◇ ((X2 ◇ X3) ◇ (X1 ◇ X0))) = X0 :=
    mod_symm (h ..)
  have eq10 : sK0 ≠ (sK1 ◇ (sK2 ◇ (sK1 ◇ sK0))) := mod_symm nh
  have eq11 (X3 X4 X5 : G) : (X4 ◇ (X3 ◇ (X4 ◇ X5))) = X5 := superpose eq9 eq9
  have eq21 : sK0 ≠ sK0 := superpose eq11 eq10
  subsumption eq21 rfl
```

FIGURE 10. Example of a proof reconstructed from output of *Vampire*. Note how the proof proceeds by contradiction and uses the `superpose` and `subsumption` steps implemented in *Lean*.

```
@[equational_result] theorem Equation3973_implies_Equation4023
    (G : Type _) [Magma G] (h : Equation3973 G) : Equation4023 G := fun x y z =>
  let v0 := M z x
  let v1 := M z v0
  let v2 := M v1 y
  let v3 := M z v1
  have h4 := R v2
  have h5 := R z
  have h6 := h z x v1
  let v7 := M x (M v1 z)
  T (T (T (T (h x y z) (h (M y v0) z v2)) (C (C h5 (C h4 (T (T (h y v0 v2)
  (C (C (T (T (T h6 (h v7 v1 v0)) (C (C (R v1) (T (h v0 v7 z) (C (S h6) h5)))
  (R v0))) (S (h z v1 v0))) (R (M v2 y))) h4)) (S (h y v3 v2))))) h4))
  (S (h (M y v3) z v2))) (S (h v1 y z))
```

FIGURE 11. Example of a proof reconstructed by *MagmaEgg*. Note the proof only uses reflexivity, symmetry, transitivity, and congruence of equality.

For saturation proofs with *Vampire*, we implemented analogs of the *superpose*, *resolve*, and *subsumption* steps in *Lean*. Proofs can then be reconstructed as sequences of these steps (and additional technicalities) as shown in Figure 10.

For equational proofs from external provers, like *MagmaEgg*, we also used a tailored version of reconstruction. Specifically, the *MagmaEgg* implementation turns *explanations* [56] from *egg* into *Lean* proofs by simple applications of the defining properties of equality as shown in Figure 11.